

CS-455: The Right Word at the Right Time

Nuh Al-Sharafi
nuh.sharafi@sabanciuniv.edu
Student ID: 32702

Musab Ahmed Khan
musab.khan@sabanciuniv.edu
Student ID: 33017

June 2026

Abstract

We built *The Right Word at the Right Time*, a terminal-based Turkish vocabulary recommender for learners who already know part of the language. The system indexes 50,000 high-frequency words from OpenSubtitles, retrieves candidates with a multilingual bi-encoder and FAISS, and reranks them with a lightweight multi-objective scorer. A separate chat mode adds an LLM tutor for explanations and follow-up questions. Benchmarks in `alpha.ipynb` over simulated learner profiles show that the weighted feature reranker consistently outperforms bi-encoder-only retrieval and a Turkish cross-encoder baseline on our evaluation metric. The best configuration (`paraphrase-multilingual-mpnet-base-v2`, mean embedding, weighted feature reranker) reached an average combined score of 0.671. The same pipeline is exposed through four CLI entry points, including a Rich-based study interface.

1 Introduction

The acquisition of a new language often presents a significant challenge for learners who have informally picked up foundational skills but lack a structured vocabulary. Traditional language learning platforms, such as Duolingo or standard textbooks, typically assume a “zero-knowledge” starting point, leading to a monotonous and repetitive experience for intermediate or informal learners. Conversely, skipping ahead often results in missing critical foundational gaps that were not covered during informal acquisition. This dichotomy highlights a pressing need for adaptive learning systems that can intelligently identify a user’s current knowledge and recommend the exact “next word” required to advance.

This project introduces “The Right Word at the Right Time,” an adaptive neural retrieval system designed specifically for Turkish language learners. While traditional methods rely on static curriculum paths, this system leverages modern advancements in Large Language Models (LLMs) and semantic embeddings to create a dynamic recommendation pipeline. By modeling the Turkish language through a RAG-like architecture, the system provides personalized vocabulary suggestions based on semantic proximity to words the user already knows, filtered by frequency data derived from the OpenSubtitles Turkish dataset.

The primary objective is to build a terminal-based application that takes a list of a user’s known words and outputs the most relevant next words to learn. Beyond simple recommendation, the system incorporates a decoder-based LLM to provide contextual definitions, translations, and example sentences, allowing for interactive clarifying questions. Because labeled “next word” data were not available, we evaluate ranking quality with intrinsic metrics on simulated learner profiles, comparing bi-encoder-only retrieval, weighted feature reranking, and a cross-encoder baseline. This approach aims to bridge the gap between informal language acquisition and organized proficiency through a scalable, neural-driven framework.

2 Literature review

Modeling an adaptive language acquisition system as a recommendation framework allows for the integration of recent computational advancements to design highly effective educational tools. The central challenge in this approach lies in understanding the individual learner’s cognitive state to optimize the pedagogical utility of the recommendations. To address this, foundational inspiration is drawn from Lev Vygotsky’s Zone of

Proximal Development[4] and Stephen Krashen’s Input Hypothesis[2], which together provide the theoretical basis for matching learners with appropriately challenging material. In addition to these theories, the work of Zhou and Wang[5] offers a compelling practical example of how dynamic learner states can be operationalized within a recommendation system for a different target language.

Lev Vygotsky’s Zone of Proximal Development (ZPD)[4] provides the psychological underpinning for adaptive instructional design. Vygotsky defined the ZPD as the cognitive distance between what a learner can achieve independently and what they can accomplish under the guidance of a “More Knowledgeable Other” or capable peers. Modern intelligent tutoring environments continuously calibrate the difficulty of vocabulary and tasks to keep the learner actively engaged in this optimal instructional “sweet spot”. By dynamically providing personalized learning paths and adjusting content difficulty in real-time, AI-powered systems operationalize the ZPD to maximize cognitive growth and prevent both cognitive stagnation and overwhelming frustration.

Complementing the collaborative scaffolding of the ZPD, Stephen Krashen’s Input Hypothesis[2] provides concrete guidance for the linguistic complexity of the recommended vocabulary. Krashen posited that second language acquisition is most effective when learners are exposed to “comprehensible input” that is slightly beyond their current level of linguistic competence, formalized as the $i+1$ level. If a learner’s current competence is i , the optimal target for new input is $i + 1$, allowing the learner to utilize context and existing knowledge to infer the meaning of unfamiliar elements without raising their “affective filter” (negative emotions that form a psychological barrier).

Providing a practical computational application of these pedagogical theories, Zhou and Wang[5] developed an online personalized English learning path recommendation system that integrates domain knowledge graphs with deep reinforcement learning. Their system captures both prerequisite and semantic relationships among knowledge points, utilizing graph-based propagation and an exponential forgetting mechanism to continuously update the learner’s mastery state based on real-time interaction feedback. By formulating the recommendation task as a Markov Decision Process and employing a combination of Q-learning for value-based pruning and Proximal Policy Optimization (PPO) for action selection, their model effectively balances dynamic learner states with structured knowledge modeling. This framework demonstrates a successful architectural blueprint for how an adaptive system can continuously optimize personalized learning trajectories in a live educational environment. Unfortunately, their system is mostly beyond our scope for this project as we will only focus on vocabulary acquisition not whole language acquisition.

3 Methodology

3.1 Data

To develop a system tailored for Turkish language learners, we sought a dataset representative of everyday spoken Turkish. Consequently, we utilized the OpenSubtitles corpus [3]. Although this dataset is known to exhibit some translation artifacts, it provides an extensive and valuable resource comprising approximately 64 million sentences of conversational Turkish.

The preprocessing pipeline was adapted from a prior personal project in which the dataset was modeled as a bigram graph and structured into a relational database. This database consisted of two tables: a node table representing unique words and an edge table representing adjacent word pairs, with frequencies serving as weights. For the current project, only the node table (comprising lowercased words and their corresponding corpus frequencies) was utilized, and no lemmatization was performed. The initial parsing and database construction required approximately seven hours.

Once that database was ported, we used a bi-encoder `paraphrase-multilingual-mpnet-base-v2` to encode the words in the word-frequency table with `min_weight` of 10 and `max_vocab_size` of 50,000. The embeddings were then stored in a FAISS inner-product index along with a word list for mapping the embeddings to the words. Additionally, a `meta.json` is generated with the configuration used to generate the embeddings; the CLI reads this file on startup so the same index is reused when parameters match.

Listing 1: Sample meta.json

```
{
  "bi_model": "sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
```

```

"min_weight": 10,
"max_vocab_size": 50000,
"num_vectors": 50000,
"dimension": 768
}

```

3.2 Design

The system follows a two-phase pipeline, illustrated in Figure 1. An offline phase is run once to build a searchable semantic index from the corpus vocabulary, and an online phase runs at each session to produce personalized word recommendations for the learner.

In the offline phase, the word-frequency database is first loaded as described in the data section above.

At runtime, the learner’s known words are maintained in a *LearnerProfile* persisted as a JSON file. Before each query, a subset of the learner’s known words is selected and aggregated into a single query vector. The default selection strategy, *diverse*, draws from the middle and lower frequency bands of the learner’s known vocabulary rather than the most common words, so that recommendations evolve as the learner acquires more vocabulary. An alternative *topic* strategy selects known words whose embeddings are closest to a user-supplied topic keyword. Once a query vector is formed, the FAISS index is searched for the $k = 200$ nearest neighbors among unknown words, producing an initial candidate pool.

The candidate pool is then reranked by the *WeightedFeatureReranker*. Drawing on the pedagogical framing in Section 2, we treat semantic proximity to known words as a proxy for comprehensible input [2], frequency and difficulty terms as a proxy for staying near the learner’s zone of proximal development rather than jumping too far ahead [4], and morphological overlap as related vocabulary in the same learning neighborhood. These objectives are combined in a weighted score (default weights $w_s=w_f=0.35$, $w_m=0.10$, $w_d=0.15$, $w_e=0.05$):

$$\text{score}(w) = w_s S + w_f F + w_m M - w_d D - w_e E \quad (1)$$

where S is bi-encoder semantic similarity, F is normalized log-frequency utility, M is morphological root overlap with known words (longest common prefix), D is a diversity term, and E is a difficulty penalty when the word is much rarer than the learner’s average. The final suggestions are chosen iteratively via Maximum Marginal Relevance (MMR) [1]: at each step the candidate with the highest $\text{score}(w)$ is picked, where D is the maximum cosine similarity between w and words already selected—so D is *subtracted* here to spread out the list.

An alternative cross-encoder reranker scores each candidate against a concatenated string of known words using `dbmdz/bert-base-turkish-cased`; benchmark results show it underperforms the weighted feature reranker on our evaluation metric.

When the LLM tutor is enabled, selected words are passed to an OpenRouter-hosted language model (by default `openrouter/free`), which provides a bilingual explanation and example sentences. Follow-up questions continue in a stateful conversation, and the word is added to the learner’s profile upon confirmation.

3.3 Interface

In addition to being importable as a Python module, the system exposes four command-line entry points corresponding to different levels of engagement. `rwrt-build-index` is a one-time setup command that encodes the vocabulary and writes the FAISS artifacts to disk. `rwrt-recommend` is a stateless command that accepts a list of known words and prints the top suggestions, optionally loading and updating a learner profile. `rwrt-learn` wraps the same pipeline in an interactive loop where the learner can mark words as known one by one without making any LLM calls, suitable for offline or low-latency sessions. All three commands share a common set of flags for controlling the vocabulary filter, the number of candidates and final suggestions, the query strategy, and the reranker weights.

The primary interface is `rwrt-chat`, a terminal user interface built with the Rich library. Upon launch, the TUI displays a ranked table of suggestions showing each word alongside its bi-encoder score, weighted-feature score (`wf`), optional cross-encoder score, and corpus frequency. The learner interacts through a single input line: entering a number such as 4 opens that word for study, while appending `k` to a number (e.g. `1k`, `2k`)

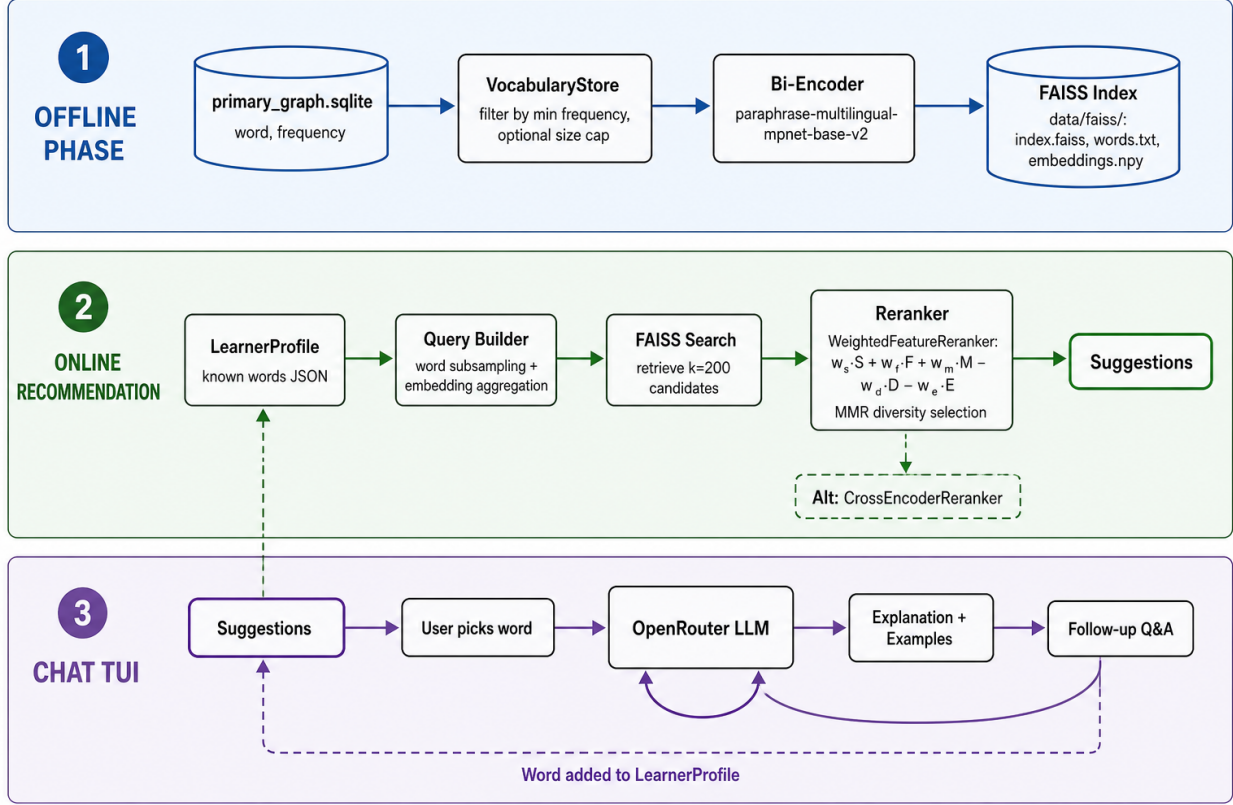


Figure 1: System architecture: offline index build and online retrieval, reranking, and optional LLM tutor.

marks it as already known without invoking the LLM. These actions can be combined, so a learner may type `1k`, `2k`, `3k`, `4` to mark three words as known and immediately study the fourth. Pressing `r` refreshes the suggestion list, and `q` exits the session.

When a word is opened for study, the LLM tutor produces an explanation in both English and Turkish with example sentences, rendered as formatted Markdown in the terminal (Figure 3). The learner can then type follow-up questions in natural language. Pressing Enter on an empty line confirms that the word has been learned, adds it to the profile, and returns to the suggestion screen. Pressing `b` returns without recording the word as learned.

4 Evaluation and Results

4.1 Evaluation setup

Our original proposal planned ranking metrics such as Mean Reciprocal Rank (MRR) and Recall@10 against held-out “next words.” In practice we did not have reliable ground-truth labels for Turkish vocabulary progression at the word level, so we adopted an *intrinsic* evaluation in `alpha.ipynb` instead.

We simulate learners by sampling known-word sets of sizes 10, 25, 50, 100, 200, and 500 (five random profiles per size), retrieve the top 10 unknown suggestions per profile, and score the resulting lists with the **same** combined formula (Equation (1)), using the same default weights. The benchmark in `alpha.ipynb` averages each term over the ten recommended words rather than computing them per candidate at pick time. In practice the terms match the design intent: S is semantic fit to known words, F is frequency utility, M is morphological relatedness, and E penalizes overly rare words. The only subtlety is D : during reranking it is subtracted as an MMR penalty (maximum similarity to words already chosen), while in evaluation it is the diversity of the final list, defined as $1 - \text{mean pairwise cosine}$, and enters with a plus sign—so a *diverse*

rwrt chat - pick a word to study, or mark words as known with k (e.g. 1k, 2k, 4 marks 1-2 known and opens 4).

Suggestions · 17 known words

#	Word	bi	wf	cross	freq
1	bunu	0.918	0.785	-	2453889
2	şeyin	0.959	0.563	-	168509
3	şeye	0.965	0.554	-	148212
4	gibi	0.926	0.537	-	2434483
5	olduğunu	0.971	0.536	-	1029337

Pick 1-5 to study, NK to mark known (e.g. 1k, 2k, 4), r refresh, q quit: 4k,

Figure 2: Suggestion table in `rwrt-chat`.

Here is an explanation of the Turkish word **human** to help you progress in your language journey.

1. English meaning

Human means "of this" or "its". It is the genitive (possessive) form of the pronoun *bu* (this).

2. Turkish explanation

"Human" kelimesi, "bu" kelimesinin ilgi (genitive) hali'dir. Bir şeyin bu nesneye veya bu kişiye ait olduğunu göstermek için kullanılır.

3. Examples

- Human gibi bir şey istiyordum. (I want something like this / like of this.)
- Human adı ne? (What is the name? / What is the name of this?)
- Bu human temalıdır. (This is human-themed. / It really looks like the color of this.)

4. Usage note

Common mistake: Do not forget that when you use **human** to show possession, the noun that follows it must take a 3rd-person possessive suffix (e.g., *human ramp* - its tail, not *human ramp*).

5. grammar

The word **human** is formed by adding the genitive case suffix to the pronoun *bu* (this).

- *bu* (this) + *-u* (buffer letter) + *-un* (genitive suffix meaning "of") → **human** (of this / its).

In Turkish, when you add a case suffix starting with a vowel to the pronoun *bu* (this), *bu* (that), or *o* (he/she/it), you must insert the buffer letter "u" in between.

Grammar in the examples:

- In "Human adı" (its name), **human** has the genitive suffix *-un*, and *adı* (name) has the 3rd-person possessive suffix *-ı*. They work together as a noun phrase.

Figure 3: LLM tutor explanation for a selected word.

Bi-encoder	Query strategy	Reranker	Avg. combined
mpnet	mean_embedding	weighted feature	0.671
mpnet	mean_embedding	bi-encoder only	0.598
mpnet	mean_embedding	cross-encoder	0.580
mpnet	inv. weighted mean	weighted feature	0.666
e5-small	mean_embedding	weighted feature	0.653
distiluse	mean_embedding	weighted feature	0.643

Table 1: Representative benchmark averages (30 profiles per row). Full grid in `alpha.ipynb`.

cluster scores higher on D in the benchmark. That is intentional for measuring variety; the reranker still uses subtraction to avoid duplicate neighbors at selection time. Each configuration is averaged over 30 profile runs; results are saved to `data/faiss_alpha/benchmark_results.json`.

We compared three bi-encoders (paraphrase-multilingual-mpnet-base-v2, distiluse-base-multilingual-cased-v2, multilingual-e5-small), three query strategies (mean_embedding, weighted_mean, inverse_weighted_mean), and three reranking modes: bi-encoder only, weighted feature reranker, and cross-encoder reranker. Cross-encoder runs were limited to the two larger models because of GPU memory and latency.

4.2 Notebook benchmark results

Table 1 summarizes representative configurations on mean combined score. The weighted feature reranker improves every bi-encoder and query-strategy pair we tested relative to bi-encoder-only retrieval. The cross-encoder reranker scores lower on the combined metric despite sometimes high S , because it does not optimize F , M , or D directly.

On the best configuration, weighted feature reranking raised the average combined score from 0.598 to 0.671 (+12.3%). Sub-metric averages for that run were $S=0.955$, $F=0.691$, $M=0.880$, $D=0.049$, and $E=0.000$. Larger simulated profiles tended to score higher (Figure 7). The smaller e5 model reached 0.653 with weighted feature reranking, suggesting a lighter deployment option at modest quality cost.

Figure 4 plots mean combined score for each bi-encoder and reranking mode (mean-embedding query strategy). Weighted feature reranking is highest for every model; the cross-encoder trails despite strong semantic scores on some runs. Figure 5 decomposes the winning mpnet configuration: high S and M with a small positive contribution from list diversity D . Figure 6 shows that weighted feature scores are relatively stable across query strategies for mpnet and e5-small, with a slight edge for mean embedding on the best model. Figure 8 relates combined score to frequency appropriateness across all runs. Figure 9 compares sub-metrics for bi-only, weighted feature, and cross-encoder on the best model and query strategy.

Qualitative checks in the notebook’s topic section (keywords such as *yemek*, *seyahat*, *okul*) showed that weighted feature suggestions often share stems with the topic word (e.g. *okula*, *okulda* for *okul*), while cross-encoder picks were less morphologically aligned.

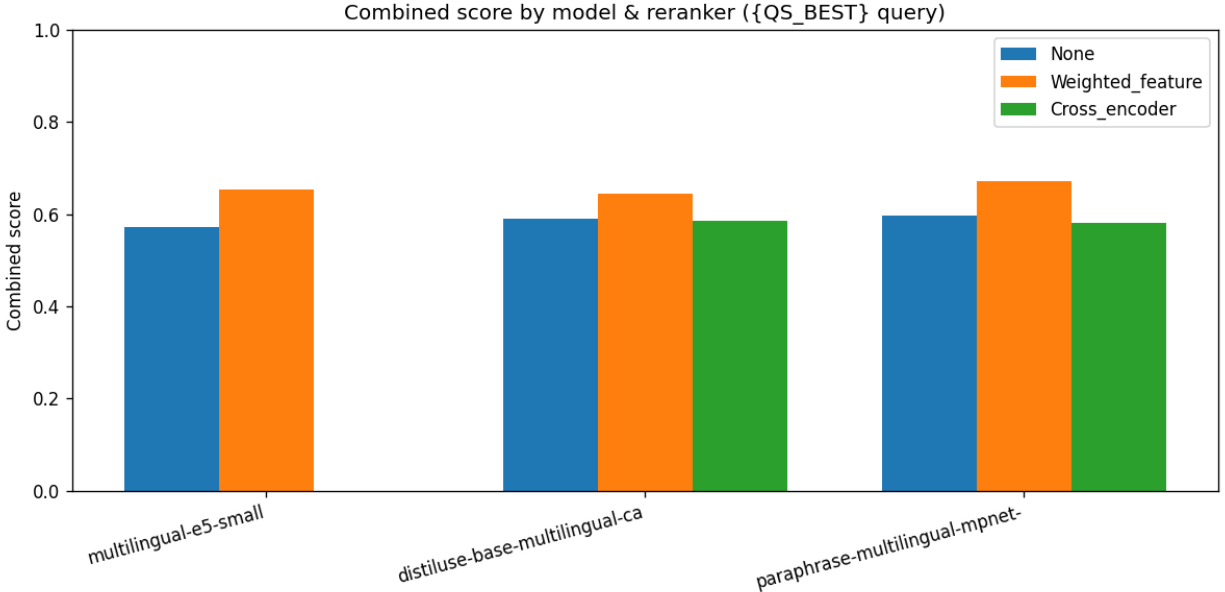


Figure 4: Mean combined evaluation score by bi-encoder and reranker (mean_embedding query), from alpha.ipynb.

4.3 CLI demonstration

The `rwrt` package mirrors the notebook pipeline in four command-line tools, all sharing `PipelineConfig` defaults and override flags.

One-shot recommend (rwrt-recommend). A quick demo: `rwrt-recommend merhaba teşekkür evet kitap -n 5` loads the cached mpnet index, applies weighted feature reranking, and prints ranked words with `bi=`, `wf=`, `cross=`, and `freq=` columns. The `wf` value is the reranking score from Equation (1) (including MMR). `cross=` is empty unless `--reranker-type cross_encoder` is set. With `--bi-only`, output follows pure FAISS order—useful for comparing against the default.

Interactive learn (rwrt-learn). Same recommender in a text loop without LLM calls; the learner picks a numbered suggestion and the word is appended to a JSON profile.

Chat tutor (rwrt-chat). The Rich TUI (Figure 2) combines recommendations with an OpenRouter-backed tutor (`.env` API key). Selecting a word triggers a bilingual explanation; follow-up questions are supported before the word is saved to the profile.

Index build (rwrt-build-index). Indexes live under `data/faiss/<model_slug>/` with `meta.json`. The CLI auto-resolves the correct subfolder when `data/faiss` is passed. A smoke script (`scripts/run_all_checks.sh`) and 88 pytest cases cover the Python API.

5 Discussion

Weighted feature reranking gave the strongest results on our evaluation metric. The cross-encoder checkpoint (`dbmdz/bert-base-turkish-cased`) is a masked-language model without a head trained for this ranking task; CLI runs with `--reranker-type cross_encoder` often surface weak or unrelated neighbors. Semantic similarity alone is not enough: learners also need appropriate frequency, morphological relatedness, and variety in the suggestion list.

The shift from MRR/Recall to intrinsic metrics was necessary given our data, but it limits direct comparison to the original proposal. Future work could add human ratings or a small labeled set of acceptable next words per profile.

The terminal interfaces evolved through several iterations: early versions required manual index paths and did not show reranker scores in the output table. The final CLI prints separate `wf` and `cross` columns so

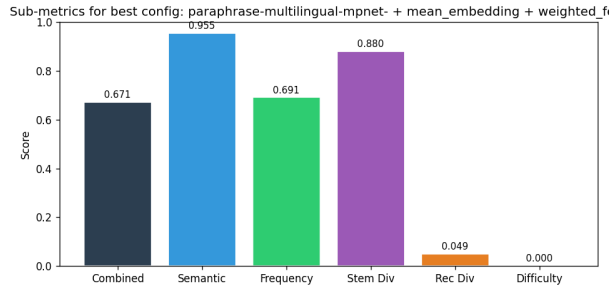


Figure 5: Sub-metric breakdown for the best configuration (mpnet, mean embedding, weighted feature).

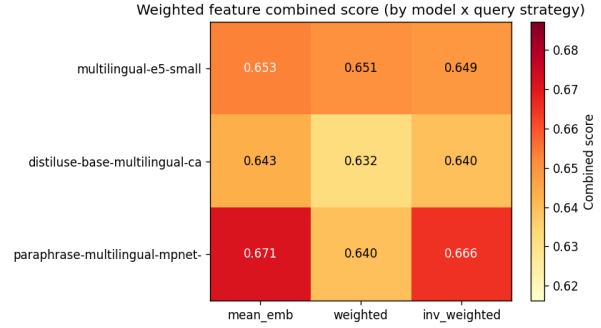


Figure 6: Weighted-feature combined score across models and query strategies.

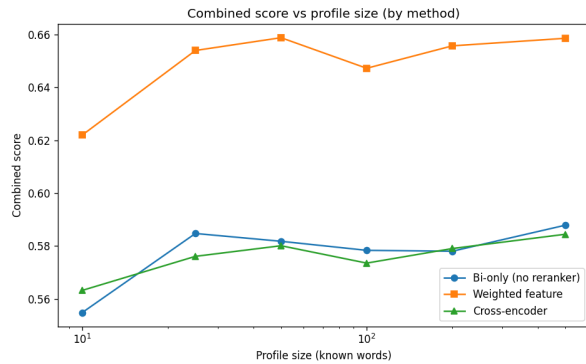


Figure 7: Combined score vs. simulated profile size (by reranking method).

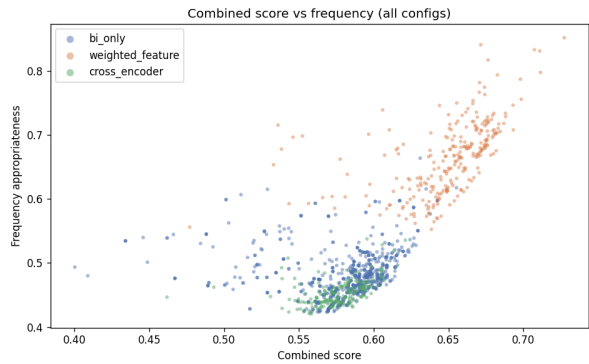


Figure 8: Combined score vs. frequency appropriateness (all configurations).

it is clear which stage produced each value.

5.1 What didn't work

One of the big design pain points was configurability where we spent 3 full days designing and re-designing the `rwrt` package until it got to a point where we were happy with it. We wanted to have a very flexible package with sensible defaults. The defaults had to be in one place. That place had to be clear, clean and small. We also wanted to have a `.env` file, that should override the defaults. We also wanted to be able to override both the `.env` and defaults using command line parameters. We wanted to be able to switch searching strategies and models on the fly, so everything had to be very standardized. The point of this was to make testing and benchmarking as easy as possible. AI helped a lot in seeing some of the ideas through which taught us where our mistakes were and allowed us to quickly iterate through solutions.

The terminal interface also went through several upgrades. Early demos could not override index paths, vocabulary caps, or reranker settings from the command line, which made side-by-side comparisons slow. Later versions added shared flags across `rwrt-recommend`, `rwrt-learn`, and `rwrt-chat`, plus automatic resolution of per-model FAISS subdirectories under `data/faiss/`.

Our original evaluation plan relied on MRR and Recall@10 against held-out next words, but we lacked a trustworthy labeled set at the word level. We moved to the intrinsic combined metric in Equation (1), with stem diversity and list-level diversity terms added after inspecting early suggestion lists that were semantically close but morphologically repetitive.

The cross-encoder path did not work as well as we hoped. The Turkish BERT checkpoint is not trained as a relevance ranker for “known words → next word,” and load-time warnings indicate a randomly initialized classification head. It remained in the project as an optional baseline (`--reranker-type cross_encoder`)

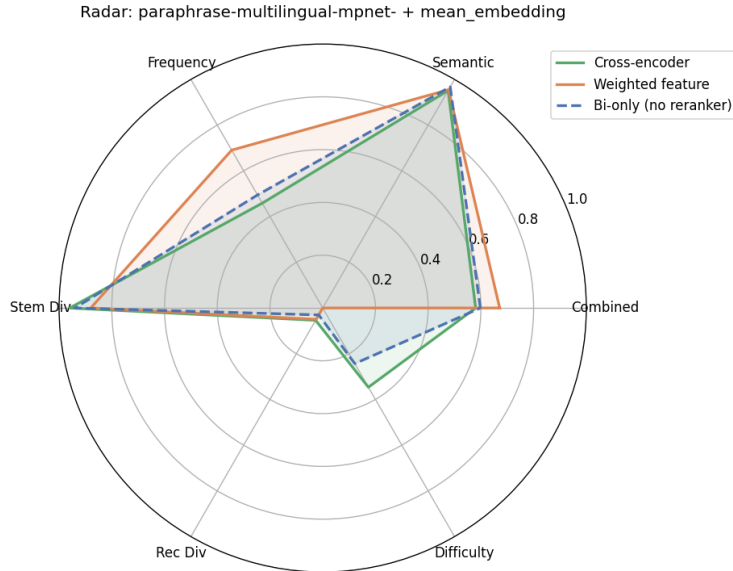


Figure 9: Radar of mean sub-metrics on the best model and query strategy: bi-encoder only, weighted feature, and cross-encoder. Cross-encoder can lead on S but loses on F , M , and D .

rather than the default.

List-level *rec diversity* (D) also caused confusion during benchmarking. We first had the sign wrong in `alpha.ipynb` (D was reporting mean pairwise similarity instead of $1 - \text{mean sim}$), which made the best runs look artificially diverse until we fixed it. After the fix, D on the winning configuration was still low (≈ 0.05), i.e. recommended words remain very close in embedding space (mean pairwise cosine ≈ 0.95). That is not a reranker bug: retrieval and reranking are built to keep suggestions near the learner’s known-word neighborhood, and high semantic coherence ($S \approx 0.95$) tends to imply low embedding spread among the top ten. MMR subtracts similarity only to words already chosen at pick time, which helps slightly on average ($D \approx 0.08$ with weighted feature vs. ≈ 0.05 bi-only) but does not optimize the same quantity as the evaluation metric. Stem diversity can still be high while D stays low, because Turkish inflections look different by prefix overlap but cluster in the same semantic region. We kept $w_d = 0.15$ as a light nudge rather than forcing unrelated words that would hurt S .

5.2 AI use

Cursor was used in the development of the `rwrt` module to speed up the process and adhere to high portability standards. All the code was thoroughly reviewed and tested to make sure it aligned with the vision and gave the expected outputs.

Opencode assistance was used to help with notebook experiment scaffolding, plotting, and drafting parts of the evaluation narrative in `alpha.ipynb`. Benchmark numbers and configuration tables were verified against saved JSON outputs.

Finally, openwebUI was used to help with rephrasing some parts of this report to ensure the tone remained professional. It goes without saying that the accuracy of the output was checked against reality before being transferred into this report.

6 Conclusion

We delivered a working Turkish vocabulary recommender with an offline FAISS index, a default multi-objective reranker, optional cross-encoder comparison, and three CLI usage modes (one-shot, interactive learn, LLM chat). The best automated configuration, mpnet, mean embedding, weighted feature reranking, achieved a combined evaluation score of 0.671 on simulated profiles, ahead of bi-encoder-only (0.598) and cross-encoder (0.580) baselines on the same metric. The system is suitable for terminal-based demos; sensible next steps are human evaluation, lemmatization-aware morphology, and a fine-tuned cross-encoder if pairwise reranking remains a goal.

References

- [1] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336. ACM, 1998.
- [2] Stephen D. Krashen. The monitor model of adult second language performance. In M. Burt, H. Dulay, and M. Finocchiaro, editors, *Viewpoints on English as a Second Language*, pages 152–161. Regents, 1977.
- [3] Pierre Lison and Jörg Tiedemann. OpenSubtitles2016: Extracting large parallel corpora from movie and TV subtitles. In Nicoletta Calzolari, Khalid Choukri, Thierry Declerck, Sara Goggi, Marko Grobelnik, Bente Maegaard, Joseph Mariani, Helene Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pages 923–929, Portorož, Slovenia, May 2016. European Language Resources Association (ELRA).
- [4] L. S. Vygotsky. *Mind in Society: Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1978.
- [5] Liu-Ying Zhou and Yuan-Yuan Wang. Simulation of personalized english learning path recommendation system based on knowledge graph and deep reinforcement learning. *Scientific Reports*, 15:34554, 2025.